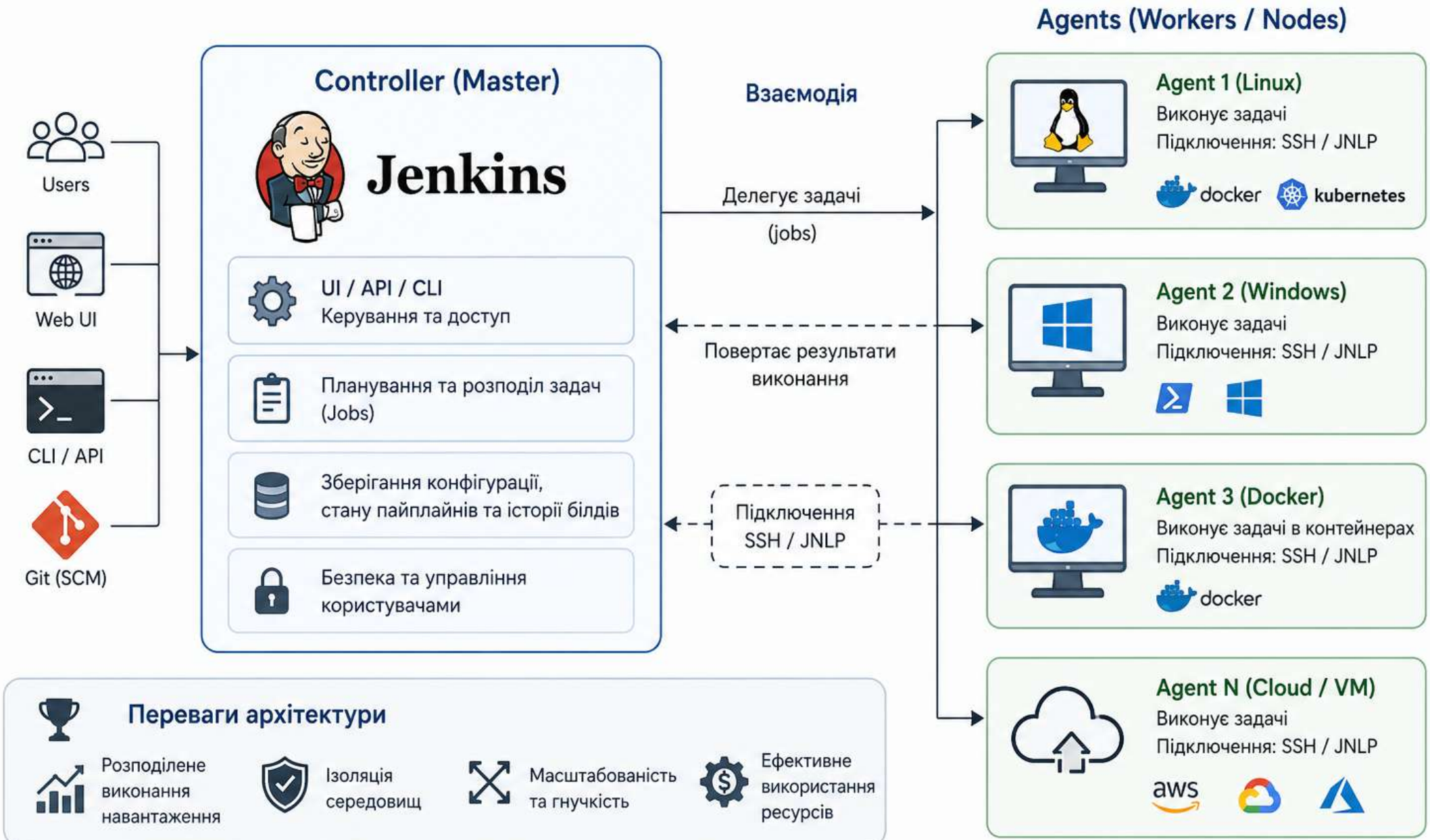
Архітектура Jenkins (Controller та Agents)

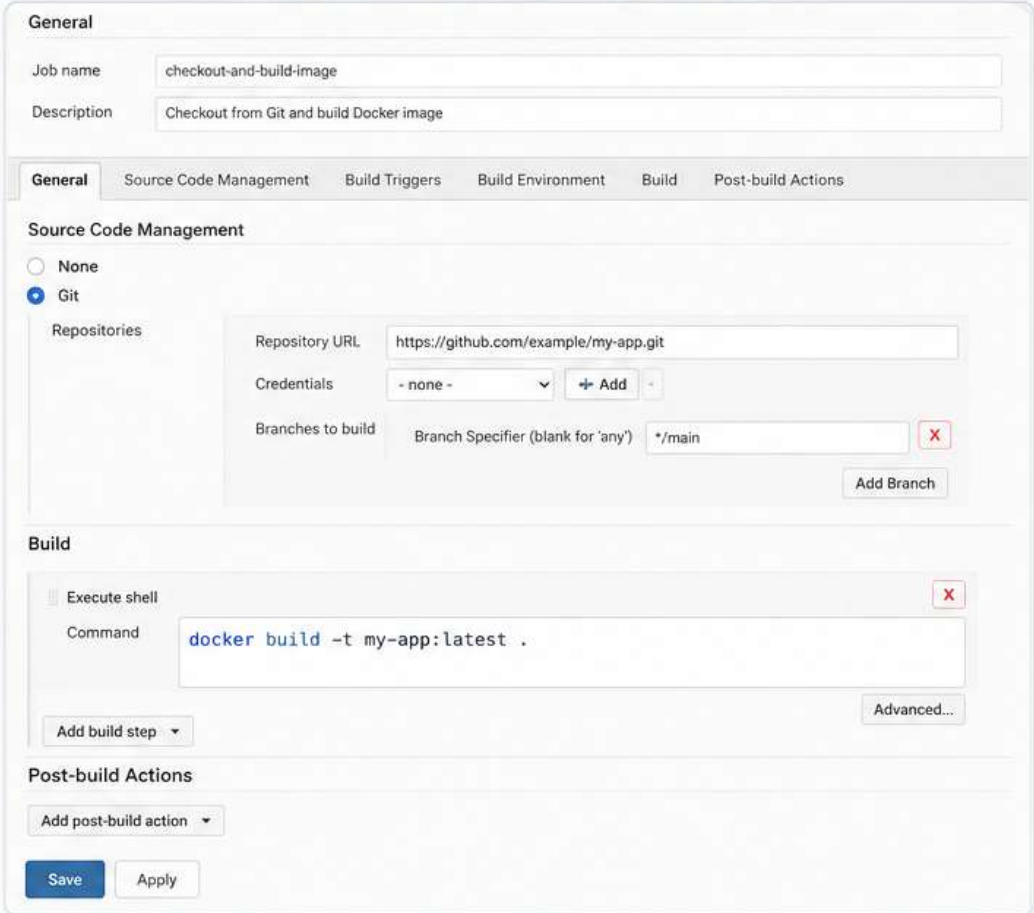


Jenkins Jobs: Freestyle vs Pipeline

🎯 **Задача:** checkout from git → build docker image

FREESTYLE JOB

Інтерфейс створення (конфігурація job)



PIPELINE JOB

Приклад Jenkinsfile (Pipeline as Code)

```
pipeline {
  agent any

  environment {
    IMAGE_NAME = 'my-app:latest'
  }

  stages {
    stage('Checkout') {
      steps {
        git url: 'https://github.com/example/my-app.git', branch: 'main'
      }
    }

    stage('Build Docker Image') {
      steps {
        script {
          sh "docker build -t ${IMAGE_NAME} ."
        }
      }
    }
  }
}
```

Pipeline Flow

Git Checkout → Build Docker Image → Success

Результат

Обидва підходи виконують одну і ту ж задачу: checkout з Git та build Docker image.

Freestyle

- Налаштовується через UI
- Добре для простих задач
- Складніше підтримувати та масштабувати

Pipeline

- Описується як код (Jenkinsfile)
- Версіонується разом з проєктом
- Легко масштабувати та підтримувати

Jenkins Best Practices для DevOps процесів

1 Infrastructure & Architecture

- [ ] Використовуй separate controller та agents
- [ ] Тримай controller “lightweight” (без важких білдів)
- [ ] Використовуй ephemeral agents (Docker/Kubernetes)
- [ ] Розділяй середовища: dev / staging / prod агенти

2 Pipeline as Code

- [ ] Використовуй тільки Jenkins Pipeline, Уникай Freestyle jobs => Всі jobs зберігай у Jenkinsfile в Git
- [ ] Version control для всіх змін pipeline

3 Security Best Practices

- [ ] Вимкни anonymous access
- [ ] Сегментуй доступ по командам/проєктах
- [ ] Мінімальні права доступу (RBAC)
- [ ] Використовуй Credentials store (не plaintext у pipeline)
- [ ] Регулярно оновлюй Jenkins і плагіни

4 Plugin Management

- [ ] Мінімальний набір плагінів (only what you use)
- [ ] Регулярна перевірка оновлень
- [ ] Уникай deprecated/experimental plugins
- [ ] Не дублюй функціональність плагінами
- [ ] Оновлюй вручну, перевіряй роботу пайплайнів після оновлення

5 Pipelines Design Principles

- [ ] Маленькі, швидкі pipeline stages. Розділяй build / test / deploy
- [ ] Використовуй паралелізацію де можливо
- [ ] Fail fast (зупинка при першій критичній помилці)

6 Performance & Scalability

- [ ] Обмежуй concurrent builds на controller
- [ ] Scale agents горизонтально
- [ ] Використовуй caching (dependencies, Docker layers)
- [ ] Очищення workspace після білду

Розробка та отладка пайплайнів

1. Засоби розробки Jenkins

- **"Configure"** -> Вбудований редактор

- **"Replay"** - для тимчасових правок. Також, щоб подивитися код, якщо пайплайн в Git

- Шпаргалка та генератор коду - **Pipeline Syntax**

- Розширення для **VSCode:**
  - Jenkins Pipeline Linter Connector
  - Jenkins Runner
  - Jenkins Jack

2. Отладка

- Завжди відкривай "Console Output" конкретного build'а. **Шукай першу помилку,** а не останню

- не покладайся на "чистоту" workspace

- перевіряй стан файлової системи:

```
pwd && ls -la
df -h ./
df -ih ./
```

3. Поради розробнику пайплайнів

- розбивай великі моноліти на декілька пайплайнів

build -> push → test на оточенні → deploy

- розбивай пайплайни на stages з розумними назвами

- boolen-значення ".toBoolean()"

- Відмінність одинарних та двойних лапок:

```
sh """
varname=1
echo \$varname
"""
```

або

```
sh '''
varname=1
echo $varname
'''
```

- shebang:

```
sh '''
#!/bin/bash
'''
```

або

```
sh '''
#!/usr/bin/env python3
'''
```

Репозіторій з демонстраційним Jenkins: https://github.com/albezuglov/jenkins-demo